

A Statistical Evaluation of Multiclass Machine Learning Models for Crime Classification

Executive Summary

This study produces multiclass machine learning models to predict crime categories using 800,000 crime observations in San Francisco between 2003 and 2015. The objective is to identify a model that provides the strongest predictive performance while maintaining robustness and interpretability.

This analysis compares three multiclass learner models: Categorical Naïve Bayes, Multinomial Logistic Regression, and Random Forest classifiers. Models were developed on the basis of four fundamental functions – a data preparation function, a data preprocessing function, a training function that trains the processed data to a predictive algorithm, and a validation function to evaluate each model using three-fold cross-validation and an independent hold-out test set. Feature engineering included temporal, geographic, and neighborhood variables, with interaction terms incorporated to capture heterogeneous effects across police districts.

Key findings:

- Multinomial Logistic Regression with interaction terms provided the best balance of predictive performance and flexibility for generalization on unseen data
- Geographic variables and police district were the strongest predictors
- Interaction terms between neighborhood and intersections reduced logistic loss, indicating spatial heterogeneity
- Random Forest model exhibited evidence of inefficiency and overfitting despite its flexibility

Recommendation

For applications requiring accurate and interpretable multiclass classification (predicting crime class given a new observation, for instance), multinomial logistic regression with interaction terms is the preferred modelling framework. Although more complex algorithms were evaluated, these algorithms exhibited limited out-of-sample performance to sufficiently justify the additional complexity.

I. Introduction

This report uses the *san_francisco_train.csv* dataset to train and evaluate multiclass classifiers (the category of crime). The dataset contains over 800,000 recorded crimes in San Francisco between January 2003 and May 2015. Each observation records the date and time of the crime, the police district (neighborhood), address, and crime category. This report aims to identify the optimal multiclass learner to predict the crime category of a new incident based on available features. Multi-class logistic loss (log-loss) will be used as the sole performance metric in this report. This metric was chosen as it sums the errors across all available classes while penalizing confident but incorrect predictions.

This report first establishes the four fundamental functions – *sf_crime*, *data_preprocessing*, *predictive_algo*, and *model_validation* – which will be called throughout the experiment. Then, three learners are trained and compared: Categorical Naïve Bayes, Multinomial Logistic Regression, and Random Forest. Lastly, the trends of crime categories are visualized against the features, and optimal features for prediction are selected.

II. Constructing Fundamental Functions

Data Preparation

The *sf_crime* function prepares the dataset and performs data transformations before the test/train splits are created. First, the data is cleaned so that column names are unified and uninformative columns (like description and resolution of the crime) are dropped. Then, date and time data are encoded as datetime features.

Next, addresses are parsed into *block*, *street_1*, and *street_2*. Splitting the *address* feature on ‘ / ’ identifies intersections and populates the *street_1* and *street_2* features. Splitting the *address* feature on ‘Block of’ identifies block addresses and populates the *block* and *street_1* features. A binary variable *intersection* is also created and set to 1 when a crime occurred at an intersection, 0 otherwise.

The encoding block numerically encodes all unique street names across *street_1* and *street_2* by collecting unique street name strings, alphabetically sorting them, and mapping them to integers via a dictionary.

Data Preprocessing

The *data_preprocessing* function applies all transformations that must be fitted on training data. To ensure reproducibility and prevent data leakage, data preprocessing was exclusively executed on the training data set. It encodes the 39 crime categories into integers, encodes the block number as block divided by 100, and builds a dictionary of street names from the training data only. All numeric features are standardized by fitting the *StandardScaler* on training data.

When *use_interaction = True* (i.e., when interaction terms are accounted for), the function creates interaction terms between the encoded (using one-hot encoding) *pddistrict* dummy variables and the *intersection* dummy. For each district column, a new interaction column is created by multiplying the two binary columns (e.g., *pddistrict_CENTRAL : intersection*). This product equals 1 when a crime is both in the specified district and at an intersection, otherwise the product equals 0. All zero interaction columns (i.e., a district with no intersection crimes) are dropped before scaling since a zero-variance column means the *StandardScaler* would have to divide by zero, which is undefined. These interaction columns are created because the effect of being at an intersection on crime type may vary across different neighborhoods. Therefore, neither neighborhood nor intersection can capture these joint effects alone.

Predictive Algorithm

The *predictive_algo* function calls the *data_preprocessing* function and trains the model on the preprocessed data using a selected learner. It returns the single most likely crime category for each row and a probability for every category for each row. This creates a matrix of shape (*n_rows*, *n_categories*) with the probabilities in each row summing to 1. Lastly, it computes two performance metrics: test accuracy and test log-loss. Accuracy is the proportion of correct predictions. Log-loss measures how well a model’s predicted probabilities match the true outcomes. It penalises the model for confident wrong predictions.

Model Validation

Model performance is validated using repeated cross-validation before evaluation on an independent hold-out test set. The *model_validation* function performs k-fold cross-validation and test set evaluation. K-fold cross-validation assesses how well the model generalises patterns, whilst the latter assesses how well the model performs on unseen data. The log-loss and accuracy from each fold are

collected and averaged to produce the k-fold scores. After cross-validation, the model is retrained from scratch on the full training set and evaluated on the held-out test set (the 40% of data never used during training or model selection). This gives the definitive performance evaluation of the model on unseen data.

III. Model Training and Evaluation

All models (each using a different multiclass learner) are evaluated using 3-fold cross-validation on the training set and a final evaluation on the test set. The sole metric this report analyses is the logarithmic-loss metric. A random guesser assigning equal probability to all 39 crime classes (categories) would achieve a log-loss of approximately $-\log\left(\frac{1}{39}\right) = 3.66$. This will be used as a natural upper baseline for comparison. Because log-loss is a cost, a successful model should lower the log-loss compared to other models.

Learner 1 – Categorical Naïve Bayes

Naïve Bayes is a probabilistic classifier derived from Bayes' theorem of conditional probabilities. We illustrate this theorem with an example below:

$$P(X = x \mid Y = c) = P(x_1|Y = c) * P(x_2|Y = c) * ... * P(x_p|Y = c)$$

$$P(\text{hour} = 10|Y = \text{Robbery}) = \frac{\text{number of robberies at hour 10}}{\text{total number of robberies}}$$

Given a set of features, Naïve Bayes estimates the probability of each class and assigns the most likely one. $P(x_j|y = c)$ is the likelihood of observing feature value x_j , given class c . The caveat of this learner is that it assumes all features are conditionally independent of one another. For example, this multiclass learner assumes that knowing the hour of the crime gives no information about the day of the week of the crime. In practice, features are most likely not perfectly independent.

The Categorical Naïve Bayes multiclass learner analyzes each crime in the training data and learns how often each crime occurs with each feature value. Then, given a new crime incident, it multiplies the learned probabilities together to produce a score for each of the 39 types of crimes and assigns the most likely crime. Because Categorical Naïve Bayes estimates $P(x_j|y = c)$ by building a frequency table, each feature must be a discrete value (e.g., longitude and latitudes won't work because they are continuous floats rather than discrete categories). Six candidate features were selected based on this criterion: *hour*, *day_of_week*, *month*, *year*, *intersection*, and *pddistrict*.

A greedy forward selection approach was then used to select each prefix (combination of features), adding one predictor at a time and measuring the change in 3-fold log-loss.

Figure 1: Categorical NB Model Selection using Greedy Forward Selection

```

# greedy forward selection to identify the optimal predictor set

nb_candidate_features = ['hour', 'day_of_week', 'month', 'year',
    ↪ 'intersection', 'pddistrict']

def nb_score(selected, k=3, nbr_loops=1):

    cv = RepeatedKFold(n_splits=k, n_repeats=nbr_loops, random_state=54321)
    lls = []

    for tr_idx, val_idx in cv.split(X_train):
        Xtr_f, Xte_f, ytr_f, yte_f, _, label_enc = nb_preprocess(
            X_train.iloc[tr_idx], X_train.iloc[val_idx],
            y_train.iloc[tr_idx], y_train.iloc[val_idx],
            selected_predictors=selected)
        nb = CategoricalNB(alpha=1.0, min_categories=2)
        nb.fit(Xtr_f, ytr_f)
        lls.append(log_loss(yte_f, nb.predict_proba(Xte_f),
            labels=np.arange(len(label_enc.classes_))))

    return np.mean(lls)

selected_nb, remaining = [], nb_candidate_features.copy()

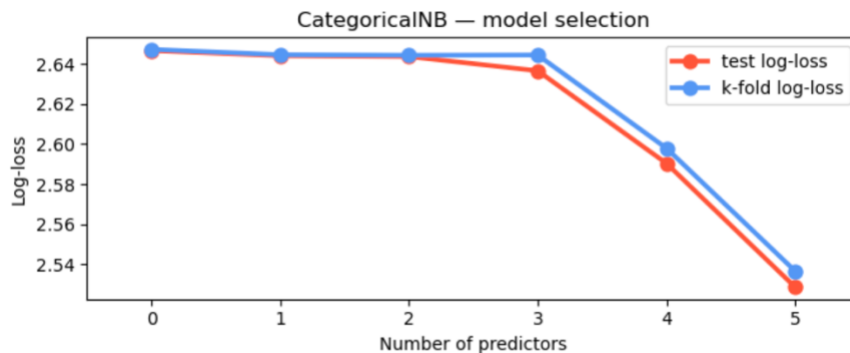
result_lst = [model_validation(learner=CategoricalNB(alpha=1.0,
    ↪ min_categories=2),

                                k=3,
                                nbr_loops=1,
                                selected_predictors=nb_candidate_features[0:m],
                                one_hot_encoding=False,
                                full_preprocessing=False,
                                scores_only=True) for m in np.arange(1,
    ↪ len(nb_candidate_features) + 1)]

```

	kf_ll	kf_acc	test_ll	test_acc	\
0	2.647209	0.202407	2.646592	0.202111	
1	2.644523	0.203191	2.644026	0.202906	
2	2.644264	0.203334	2.643620	0.203170	
3	2.644404	0.204976	2.636549	0.205400	
4	2.597959	0.215483	2.590411	0.215075	
5	2.536712	0.235123	2.528697	0.235455	

	predictors
0	[hour]
1	[hour, day_of_week]
2	[hour, day_of_week, month]
3	[hour, day_of_week, month, year]
4	[hour, day_of_week, month, year, intersection]
5	[hour, day_of_week, month, year, intersection, ...]



From Figure 1, the k-fold log-loss and test log-loss track closely at every predictor count, indicating that the model generalizes without evident overfitting. The CategoricalNB model with all six features achieved a test log-loss of 2.527, whilst the steep decline in log-loss begins at the addition of the fifth feature (police district). This suggests that the neighborhood and location type are the primary drivers of crime category prediction, whilst temporal factors individually contribute at a weaker level.

Learner 2 – Multinomial Logistic Regression

Multinomial logistic regression estimates which of the 39 crime categories a given incident belongs to. For each class c (in this case, category of crime), the model computes a linear score where $\beta_{c,p}$ are the coefficients for each feature in the feature vector x (hour, day, month, year, longitude (x), latitude (y), intersection, block, date-time, and neighborhood) in crime class c . These scores are then converted into probabilities using the softmax function.

$$z_c = \beta_{c,0} + \beta_{c,1}x_1 + \beta_{c,2}x_2 + \dots + \beta_{c,p}x_p$$

Without interactions, the coefficient for *intersection* is the same for every district. When interaction terms are added, the model learns an additional effect (coefficient) that captures how being at one intersection modifies crime probabilities from a different intersection.

Figure 2: Multinomial Logistic Regression Model Selection without Interaction Terms

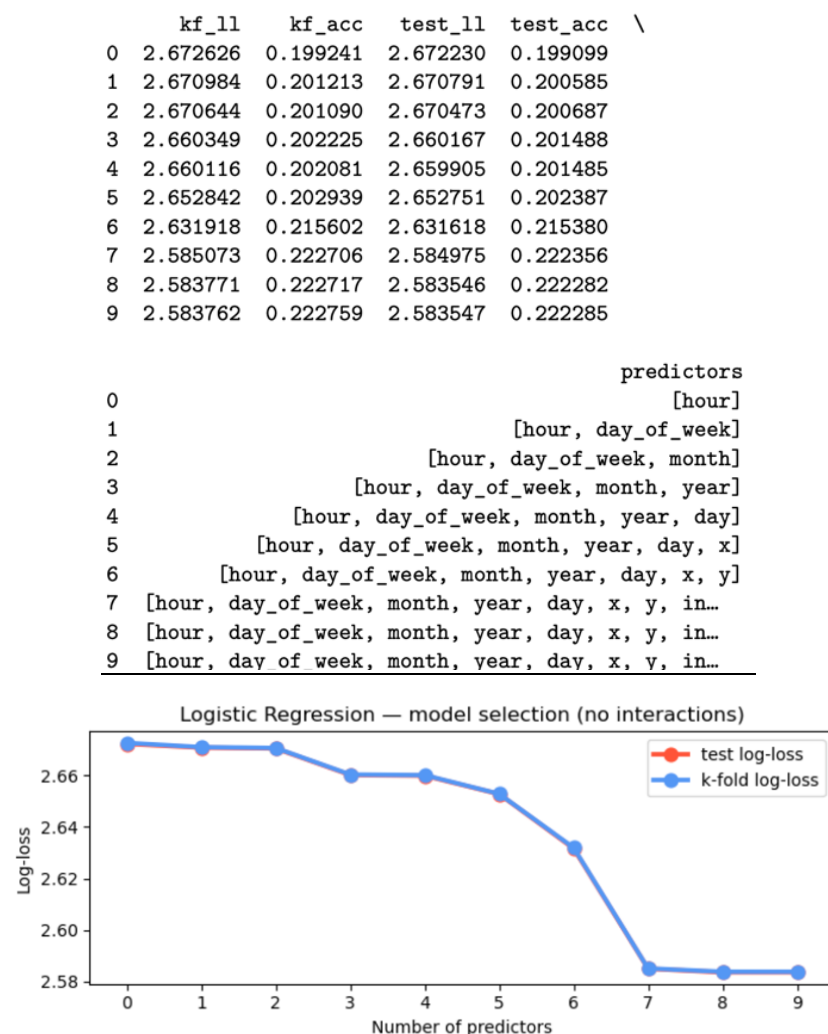
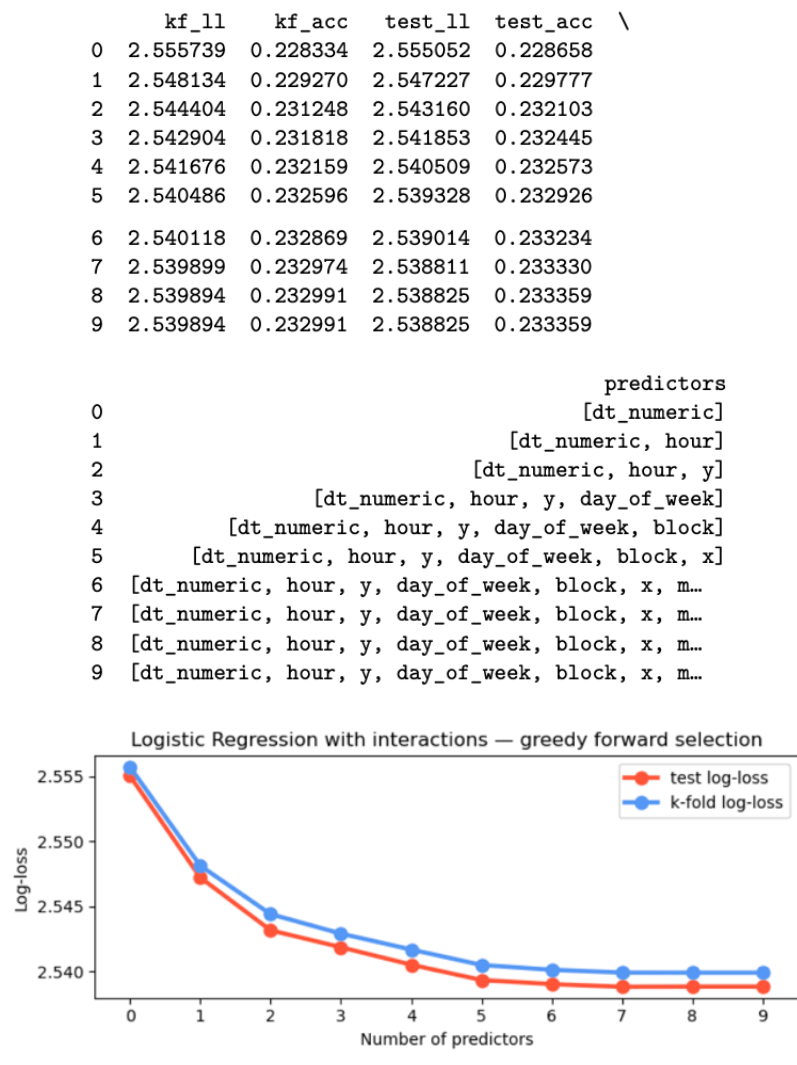


Figure 2 shows the results of a multinomial logistic regression model without interaction terms. This line graph shows marginally lower log-loss before the longitudinal and latitudinal features were added (x and y). It also reflects that the steepest decrease in log-loss occurs after the eighth feature (*intersection*) was added. Afterwards, the log-loss plateaus at around 2.59-2.58. This aligns with the analysis produced by the CategoricalNB learner – that log loss steeply declines after the *intersection* term is added. Adding the features *block* and *dt_numeric* adds virtually no further improvement.

Moreover, adding the interaction terms between neighborhood and intersections reduces log-loss by 0.045. This implies that the effect of being at an intersection varies across different police districts. Allowing coefficients to vary spatially enables the model to capture localized crime patterns that cannot be represented using additive effects alone.

Figure 3: Multinomial Logistic Regression Model Selection with Interaction Terms



In the regression with interaction terms, the *dt_numeric* feature is selected first, suggesting that it captures the overall long-run trend across the 12-year dataset. Improvements are more gradual in this regression and evenly distributed across predictors compared to the model without interaction terms. The log-loss steadily drops from 2.555 to 2.539 across the first seven predictors, with no drastic decrease between features. The log-loss plateaus after 7 predictors, indicating that adding *day*, *year*, and *intersection* produces virtually zero improvement. This is because the improvements in log-loss that the *intersection* term brings are captured in the interaction terms since the *data_preprocessing*

function includes all *pddistrict* dummy columns when *interaction = True*. The improvement in log-loss between the logistic regression models with and without interaction terms implies that the joint combination of neighborhood and intersection carries a predictive signal that neither feature alone can express in a linear model.

Learner 3 – Random Forest

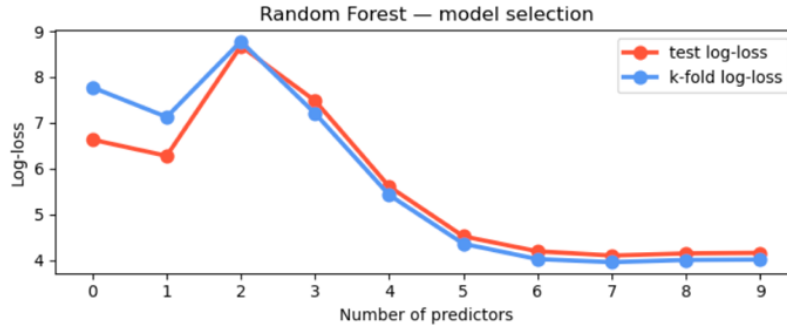
Random forest is a multiclass learner that is an ensemble of decision trees. A singular tree works by repeatedly splitting the data on the feature that best separates the classes at each node. A single decision tree can often overfit data as it memorizes the training data rather than learning generalizable patterns. A random forest of decision trees means that each tree is trained differently, and then probability estimates are averaged.

In this context, for a new crime incident, the model passes it through 200 trees simultaneously. Each decision tree assigns its own probability distribution across the 39 crime categories based on the training observations, and the 200 probability distributions are averaged to produce the final prediction. Random forest can capture non-linear interactions because each tree splits on combinations of features independently. This implies that it can capture the interactions between districts and intersections without explicitly adding columns of interaction terms.

The k-fold and test log-loss metrics for random forest with and without one-hot encoding were identical, resulting in a k-fold log-loss of 4.219 and a test log-loss of 4.374. This suggests that the representation of *pddistrict* (as one integer column or ten binary columns using one-hot encoding) is irrelevant to how the random forest learns.

Figure 4: Random Forest Model Selection

	kf_ll	kf_acc	test_ll	test_acc	\
0	7.772999	0.269825	6.631310	0.275450	
1	7.127485	0.270537	6.277986	0.275803	
2	8.777183	0.243010	8.674930	0.254091	
3	7.206453	0.241448	7.481956	0.254626	
4	5.423549	0.253285	5.608107	0.266329	
5	4.357916	0.274675	4.520937	0.287423	
6	4.022799	0.276693	4.192627	0.290390	
7	3.957733	0.281781	4.100988	0.295536	
8	4.004635	0.290418	4.146874	0.301808	
9	4.011950	0.286897	4.158358	0.299929	
					predictors
0					[x]
1					[x, y]
2					[x, y, hour]
3					[x, y, hour, day_of_week]
4					[x, y, hour, day_of_week, month]
5					[x, y, hour, day_of_week, month, year]
6					[x, y, hour, day_of_week, month, year, day]
7					[x, y, hour, day_of_week, month, year, day, in...
8					[x, y, hour, day_of_week, month, year, day, in...
9					[x, y, hour, day_of_week, month, year, day, in...



The random forest yielded the only non-monotonic pattern in the three types of learners used in this report. Adding *hour* as the third regressor drastically increased the k-fold and test log-losses. Subsequently, adding each feature improved the log loss until it started slightly increasing again after adding *block* as the ninth feature. This non-monotonic pattern likely happened because random forests are sensitive to the interaction between features in how trees are built. Adding *hour*, before geographic context is established, may have caused trees to create splits that produce very small leaves for rare crime categories. This worsens the log loss by making zero-probability predictions worse. The log-loss values reaching a height of 8.777 at 3 features reflect the zero-probability problem when trees are shallow and produce unequal leaf sizes.

Using the random forest learner, the optimal number of regressors is 8: longitude, latitude, hour, day of the week, month, year, day, and intersection. The k-fold log-loss with 8 regressors is 3.958, with a test log-loss of 4.101. Recall that this log-loss is larger than the upper baseline established by the random guesser (3.66). The relatively large difference between k-fold and test log-loss compared to near identical log-losses produced by the logistic regressions implies that the random forest is overfitting. This model learns patterns from the training data but does not fully generalize them to the held-out test set.

Summary of Results

Table 1: Summary of Results

Model	K-fold log-loss	Test log-loss	Number of Features	Features Selected
Categorical Naïve Bayes	2.537	2.529	6	<i>hour; day_of_week, month, year, intersection, pddistrict</i>
Logistic Regression without interactions	2.584	2.584	8	<i>hour; day_of_week, month, year, day, x, y, intersection</i>
Logistic Regression with interactions	2.539	2.539	8	<i>dt_numeric, hour, y, day_of_week, block, x, month, day</i>
Random Forest	3.958	4.101	8	<i>x, y, hour; day_of_week, month, year, day, intersection</i>

As seen in Table 1, the k-fold and test log-loss are nearly identical for the logistic regression models with and without interaction terms, whilst the categorical naïve Bayes model has the lowest k-fold and test log-loss metrics. The random forest model produced the largest log-loss metrics as well as the largest deviation between k-fold and test log-loss metrics. According to these metrics and the experiment procedure, Table 2 provides a qualitative summary of the three models evaluated.

Table 2: Qualitative Model Evaluation

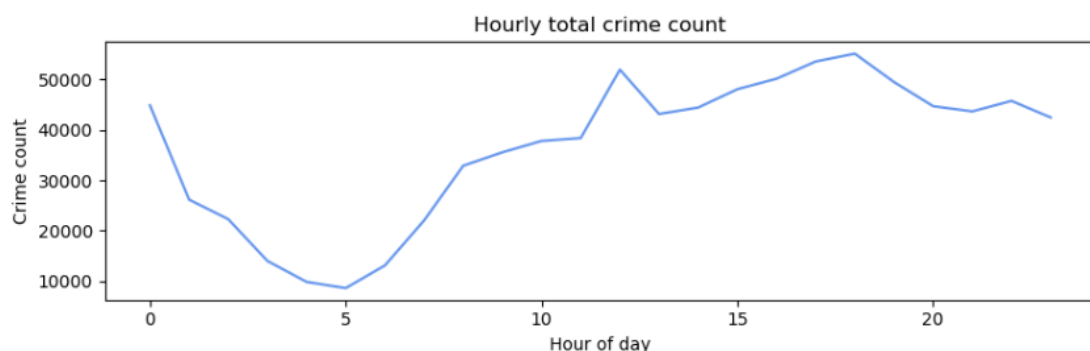
Model	Strengths	Weaknesses
Categorical Naïve Bayes	- Fastest computation (<5 minutes) - Low k-fold log-loss - Low test set log-loss - Low deviation between k-fold and test log-loss	- Based on assumption that all features are independent - Features must be discrete values
Logistic Regression	- Low k-fold log-loss - Low test set log-loss - Virtually no deviation between k-fold and test log-loss	- Decision boundaries are linear - Not suited for nonlinear relationships
Random Forest	- Decision boundaries can be irregular - Suited for nonlinear relationships	- Slowest computation (approx. 35 minutes) - High risk of overfitting - High k-fold log-loss - High test set log-loss

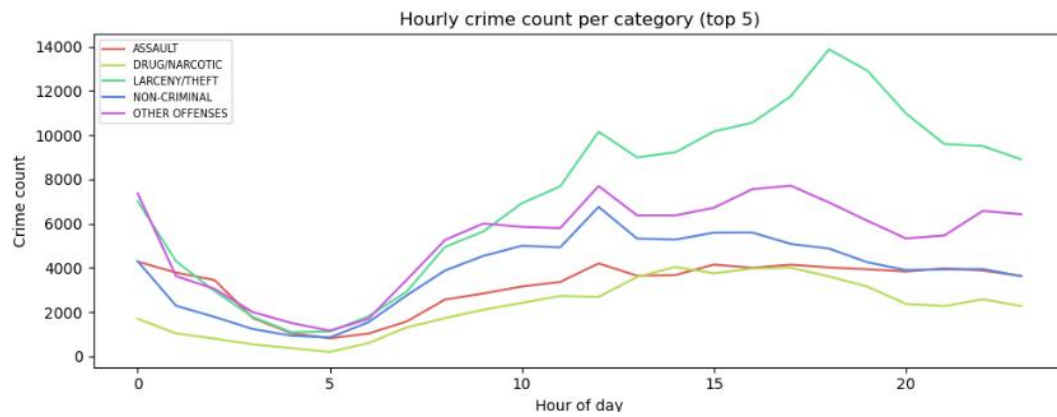
Through the experiment, we found that the categorical naïve Bayes model, although the fastest model in computation time, is limited as it bars all continuous variables such as longitudinal and latitudinal coordinates. The logistic regression model has the lowest deviation between k-fold and test set log-loss. This indicates a high ability for generalization when used as a predictive model. A limitation of this model is that the logistic regression model makes linear decision boundaries. This can be partially mitigated by incorporating interaction terms, which substantially improved predictive performance while preserving interpretability and improving decision boundary flexibility. Lastly, the random forest model was the most flexible model, as it has the capacity to create highly irregular decision boundaries by combining tree splits. However, a major drawback of this flexibility is that it introduces a high risk of overfitting, with high deviations between k-fold and test set log-loss metrics. Therefore, this report concludes that logistic regression is the optimal predictive model due to its interpretability.

IV. Data Visualization

Temporal Patterns

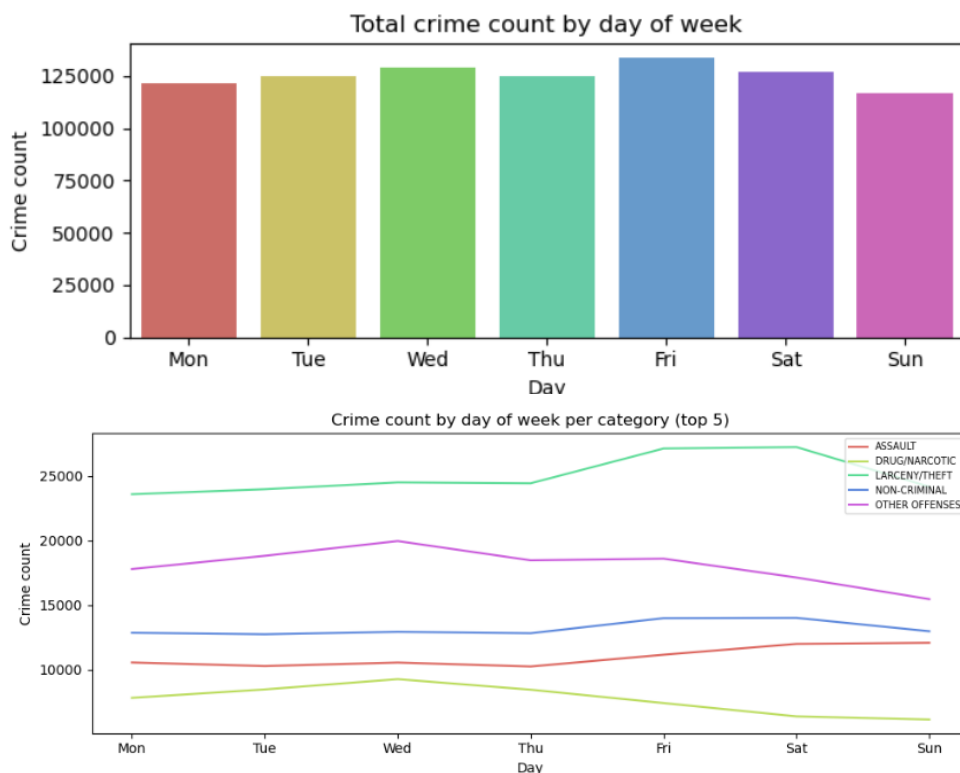
Figure 5: Hourly Changes in Crime Count





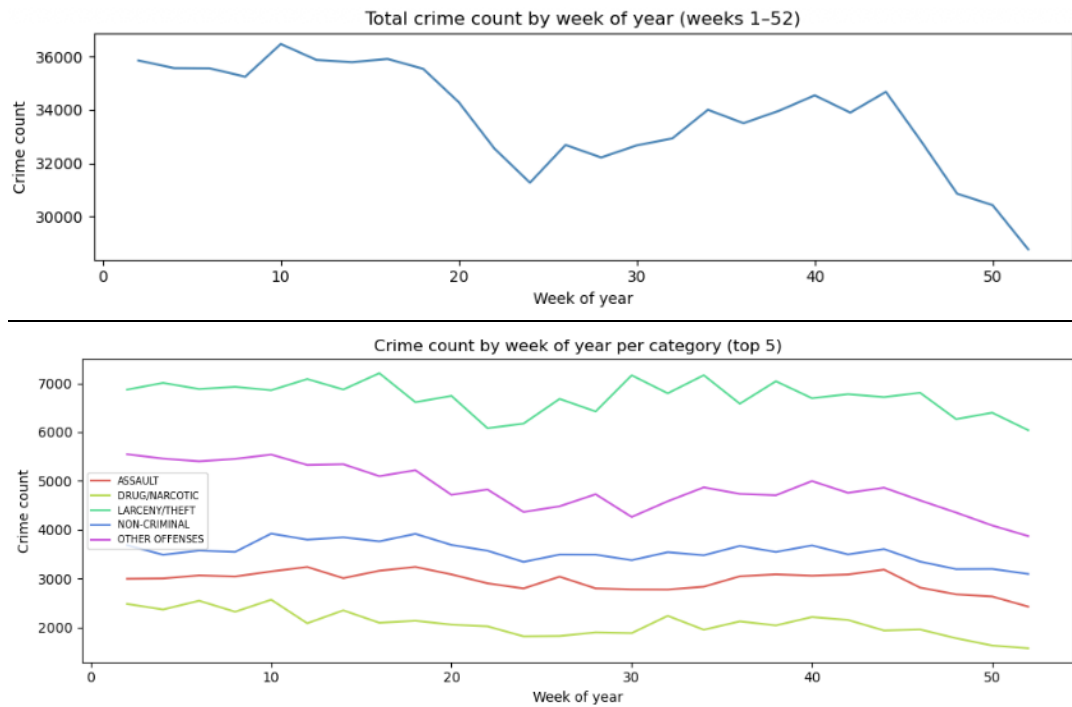
The graphs in Figure 5 illustrate a U-shaped pattern with a trough around 5 AM and rising steeply before peaking at around 12 PM and subsequently dipping around 5 PM. This suggests that crime generally has two peaks: one during daylight between 12 PM and 5 PM, and another peak leading up to 12 AM. Figure 5 also shows that larceny/theft crimes tend to follow this bimodal trend the closest, whilst all crime categories vaguely follow this trend.

Figure 6: Daily Changes in Crime Count



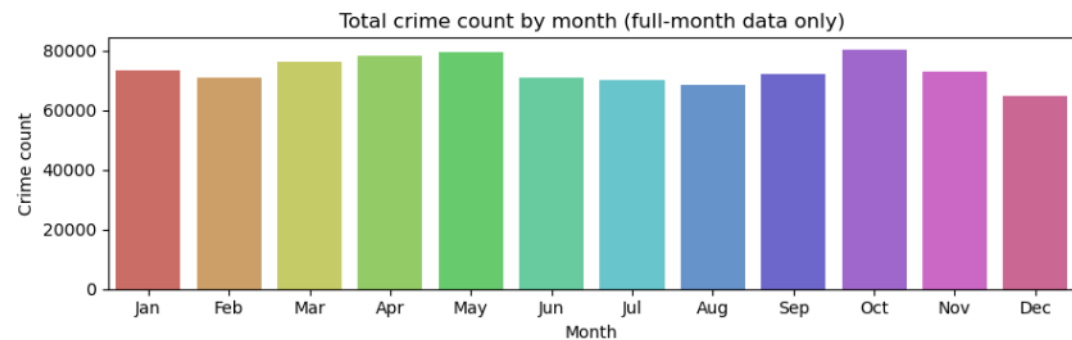
The bar chart in Figure 6 shows a relatively flat distribution of crime across all seven days, with Friday having the highest count and Sunday the lowest. The line chart shows a divergence in trends of crime – assault cases are more likely to increase towards the end of the weekend, whereas drug/narcotic and larceny/theft crimes tend to decrease towards the end of the weekend.

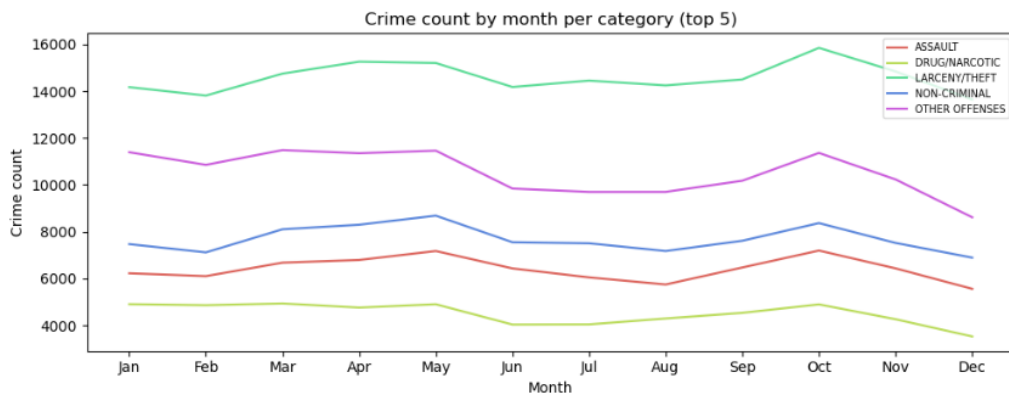
Figure 7: Weekly Changes in Crime Count



The weekly patterns in Figure 7 show noticeable variation across the year. Crime starts relatively high in January before dipping around week 10. Crime then increases again between weeks 20-30, before dropping sharply around the last weeks of the year. This illustrates that crime experiences seasonal variation. Furthermore, larceny/theft has the largest count of crimes recorded throughout the year, and, although seasonal changes are marginal, all categories of crime generally reach their lowest around week 20, before increasing for the next 20 weeks and decreasing towards the end of the year. The modest seasonality of all categories is consistent with the observation of *week_of_year* having extremely marginal effects on log-loss.

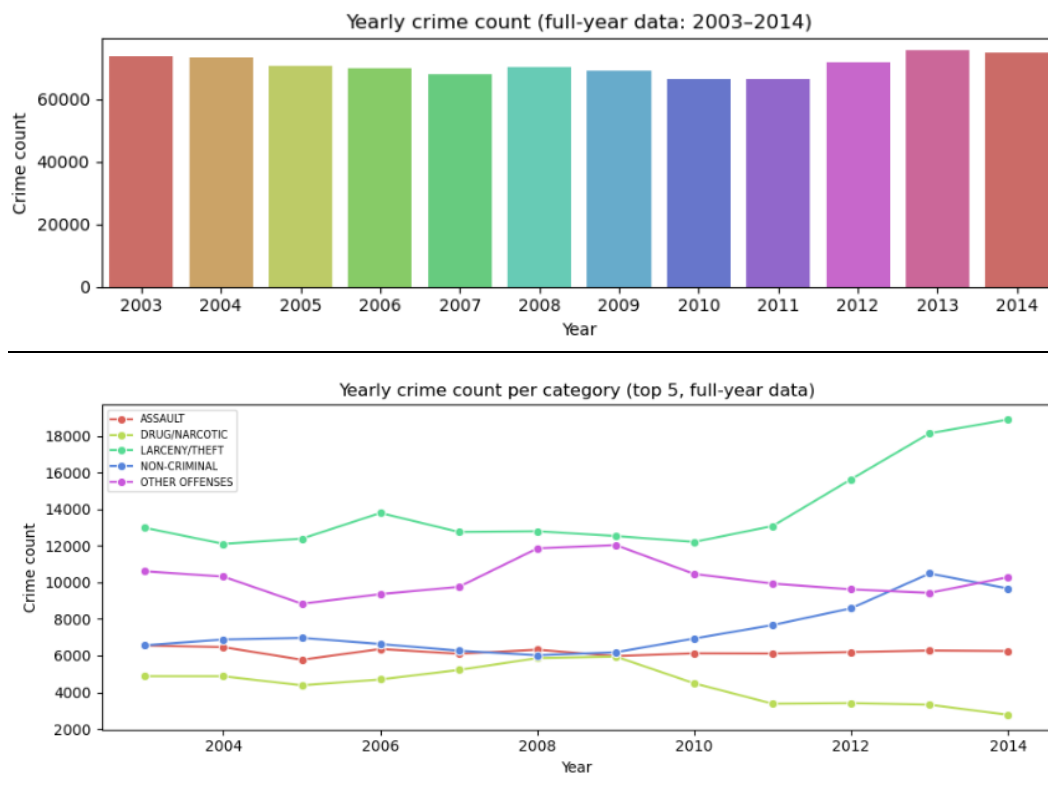
Figure 8: Monthly Changes in Crime Count





Larceny/theft shows the strongest seasonal signal, rising from around 11,000 in the winter months to around 15,000 between April and June. This follows the general seasonal pattern of crimes by month, where crime falls in the months between June and August, peaking in April, May, and October. This makes *month* a regressor that primarily helps the models to distinguish larceny from other categories.

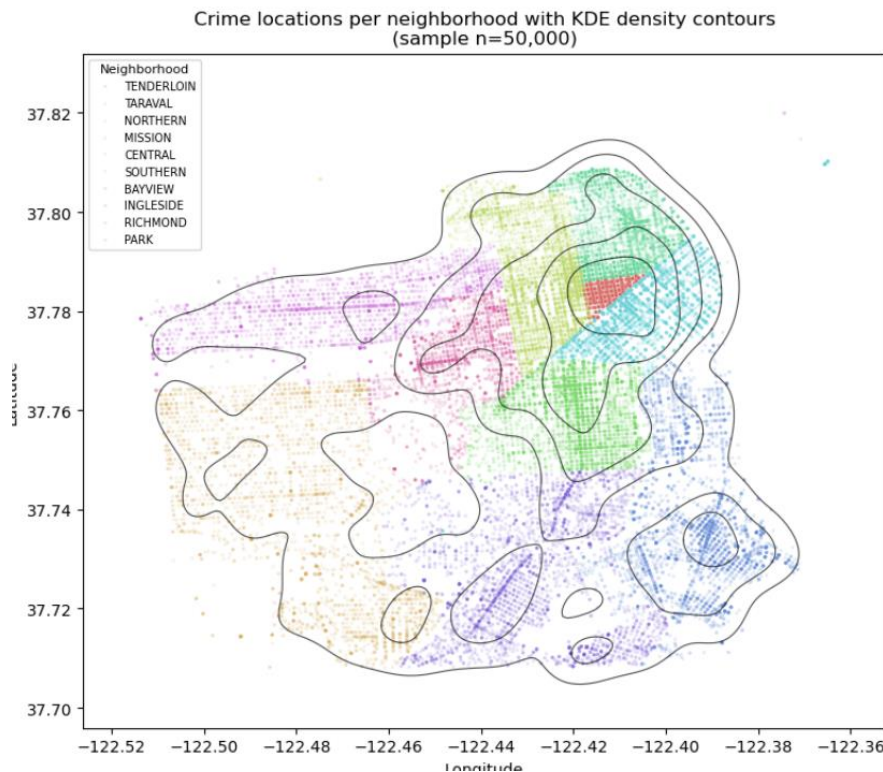
Figure 9: Yearly Changes in Crime Count



The bar chart shows that crime counts are broadly stable across the 12-year period, with no clear long-run trend. However, the line chart in Figure 9 illustrates why *year* and *dt_numeric* are useful predictors. Larceny/theft generally trends upwards, drug/narcotic crimes show a negative trend, and assault is broadly constant throughout the years. These diverging trends help explain why *dt_numeric* is selected first in the greedy forward selection with interaction terms, as it captures the long-run compositional shift in crime types across the dataset.

Geographical Distribution

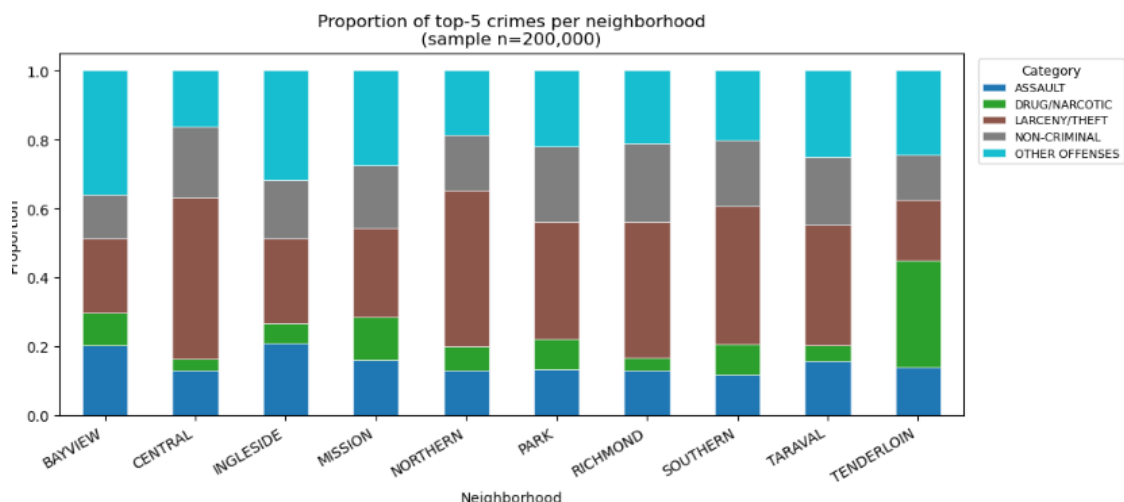
Figure 10: Crime Locations Per Neighborhood and Geographical Density



The scatterplot reveals that crime is heavily concentrated in a central band running from roughly longitude -122.46 to -122.40 and latitude 37.76 to 37.79. This corresponds to the Tenderloin, Southern, and Mission districts. The geographical clustering explains why x and y together produce the largest single log-loss improvement in the logistic regression, and why the random forest started with x and y as its first two features.

Composition by Neighborhood

Figure 11: Top 5 Crimes Per Neighborhood



The stacked bars reveal the variation in crime composition across districts – Central has larceny as the largest proportion of crimes, whilst Bayview has other offenses as the largest proportion of crimes – which justifies why *pddistrict* is an extremely impactful single predictor in the Naïve Bayes model and why it is always included in the logistic regression and random forest models.

V. Conclusion

Logistic regression provided the strongest balance between predictive accuracy and interpretability. Although more flexible algorithms were evaluated, the evidence suggests that the additional complexity did not improve predictive performance on unseen data. Therefore, for applications where reliable prediction is a priority, logistic regression is the preferred framework.

The multinomial logistic regression multiclass learner is the most optimized model because of its virtually negligible deviations between k-fold and test log-loss metrics, indicating adequate generalization of the fitted training model on unseen data. In particular, the logistic regression with interaction terms is the most optimized model because it acknowledges the fact that whether a crime occurred at an intersection has different effects on the category of crime depending on the neighborhood of the intersection. Indeed, the logistic regression with interactions improved the log-loss from 2.584 to 2.539, which confirms that the joint combination of police district and intersection type carries a predictive signal that neither feature alone can express in a linear model.

The CategoricalNB model achieved a nearly identical test log-loss of 2.529 using only six discrete features – hour, day of week, month, year, intersection, and police district – demonstrating competitive performance with a much simpler model structure. However, this model excludes continuous features like geographical coordinates. Conversely, the random forest model achieved the highest log-loss cost, exceeding the baseline of 3.66, making it a sub-optimal learner model compared to the logistic regression model.

Across the three learners, geographical location, police district, and year are the strongest predictors of each crime category, illustrated by the steep decline in log-loss metrics after these features are added to the regressor lists. This observation is strongly supported by the data visualizations, illustrating that different crime categories are generally spatially concentrated. However, it is worth noting that this analysis assumes that historical crime patterns remain representative of future observations. Moreover, this dataset excludes variables like weather, policing intensity, demographics, socioeconomic variables like income and education, etc., all of which may improve predictive performance.

Overall, these findings demonstrate that relatively simple statistical models may outperform more computationally intensive machine learning methods when prediction requires interpretability and generalization to unseen data. Organizations using predictive classification systems should prioritize feature engineering and cross-validation metrics instead of blindly adopting more complex algorithms. As an extension of this study on crime prediction, this algorithm can be improved by using datasets that include socioeconomic explanations of crime occurrence, like literacy rate and cost of living metrics.

VII. Appendix

A. Sf_Crime

```
def sf_crime(full=False,
            train=True,
            split_seed=12345,
            encoding=False,
            datetime_index=True):

    path_to_file = '/Users/nancyxu/Desktop/san_fransisco_crime_train.csv'
    X = pd.read_csv(path_to_file, header=0)

    #####
    ### Some data cleansing ###
    #####

    X.columns = X.columns.str.strip().str.lower()
    X = X.rename(columns={'dates': 'date_time'})

    X = X.drop(columns=['descript', 'resolution'])

    # String data cleaning.
    fun_dct = {'category': lambda x: x.strip(), 'pddistrict': lambda x: x.
    ↪strip()}
    X[['category', 'pddistrict']] = X.transform(fun_dct)

    slicing = X['address'].str.contains('/') & (~X['address'].str.contains(' /_
    ↪'))
    X.loc[slicing, 'address'] = X.loc[slicing, 'address'].str.replace('/', '').
    ↪str.strip()

    # Discarding unreliable data (very few rows)
    X = X.loc[X['x'] < -122, :]

    #####
    ### Feature engineering ###
    #####

    # Handling date&time data
    X['date_time'] = pd.to_datetime(X.date_time) # Encoding 'date_time' as a
    ↪datetime feature.
    X['year'] = X.date_time.dt.year
    X['month'] = X.date_time.dt.month
    X['day'] = X.date_time.dt.day
    X['hour'] = X.date_time.dt.hour
    X['minute'] = X.date_time.dt.minute
    X['second'] = X.date_time.dt.second
    X['dt_numeric'] = (X.date_time - pd.to_datetime('2003-01-06 00:01:00')).dt.
    ↪total_seconds()
    X['day_of_week'] = X.date_time.dt.weekday
    X['days_in_month'] = X.date_time.dt.days_in_month
    X['day_of_year'] = X.date_time.dt.day_of_year
    X['week_of_year'] = X.date_time.dt.isocalendar().week

    # Parsing address

    #####
    # parse into block, street_1, and street_2 (street_2 = none if not
    ↪intersection)
    mask_int = X['address'].str.contains(' / ', na=False)

    # intersection addresses
    int_ads = X.loc[mask_int, 'address'].str.split(' / ', n=1, expand=True)
    X.loc[mask_int, 'street_1'] = int_ads[0].str.strip().values
    X.loc[mask_int, 'street_2'] = int_ads[1].str.strip().values
```

```

X.loc[mask_int, 'block'] = None

# block address
blk_ads = X.loc[~mask_int, 'address']

split_parts = blk_ads.str.split('Block of', n=1, expand=True)

X.loc[~mask_int, 'block'] = pd.to_numeric(split_parts[0].str.strip(),
errors='coerce')
X.loc[~mask_int, 'street_1'] = split_parts[1].str.strip().values
X.loc[~mask_int, 'street_2'] = None

# dummy predictor =1 if crime occurred at intersection
X['intersection'] = mask_int.astype('int8')
#####

# Encoding of categorical variables.
if (encoding == True):

    # Encoding block, street_1, street_2

    #####
    # numerically encode street_1 and street_2 using dictionary
    all_streets = pd.concat([X['street_1'].dropna(), X['street_2'].
dropna()]).unique()
    street_dict = {s:i for i, s in enumerate(sorted(all_streets))}
    X['street_1'] = X['street_1'].map(street_dict)
    X['street_2'] = X['street_2'].map(street_dict)
    X['street_1'] = X['street_2'].fillna(0)
    X['street_2'] = X['street_2'].fillna(0)
    X['block'] = pd.to_numeric(X['block'], errors='coerce')
    #####

    # Controlling the type of certain features.
    features = ['minute', 'second', 'hour', 'day', 'month', 'day_of_week',
days_in_month', 'week_of_year']
    X[features] = X[features].astype('int8')

    features = ['street_1', 'street_2', 'day_of_year', 'year']
    X[features] = X[features].astype('int16')

# Setting date_time as row-index.
if (datetime_index == True):
    X = X.set_index(keys='date_time', drop=False)
    # X.index = X['date_time']
    X.index.name = 'key1'

```

```

cols_to_drop = ['dayofweek', 'days_in_month', 'second']
X = X.drop(columns=cols_to_drop)

if (full == True):
    X = X.sort_values(by=['date_time'])
    return X

else:
    y = X['category']
    X_train, X_test = train_test_split(X, test_size=.4,
random_state=split_seed, stratify=y)

    if (train == True):
        X_train = X_train.sort_values(by=['date_time'])
        return X_train

    else:
        X_test = X_test.sort_values(by=['date_time'])
        return X_test

```


B. Data Preprocessing

[illegible]

```

drop='first' if (drop_first == True)
else None,
dtype='int8')

else:
    categorical_enc = OrdinalEncoder(handle_unknown='use_encoded_value',
                                     unknown_value=np.nan, dtype='float64')

    categorical_enc.fit(X_train[['pddistrict']])
    X_train[categorical_enc.get_feature_names_out()] = categorical_enc.
    transform(X_train[['pddistrict']])
    X_test[categorical_enc.get_feature_names_out()] = categorical_enc.
    transform(X_test[['pddistrict']])

    if (one_hot_encoding == True):
        X_train = X_train.drop(columns=['pddistrict'])
        X_test = X_test.drop(columns=['pddistrict'])

        if (use_interaction == True):

            #####
            pddistrict_cols = [c for c in X_train.columns if c.
            startswith('pddistrict_')]
            for col in pddistrict_cols:
                interact_col = col + ':intersection'
                X_train[interact_col] = (X_train[col] *
                X_train['intersection']).astype('int8')
                X_test[interact_col] = (X_test[col] * X_test['intersection']).
                astype('int8')

            # filter zero-variance interaction columns
            interact_cols = [c for c in X_train.columns if ':intersection' in c]
            non_zero = [c for c in interact_cols if X_train[c].var() > 0]
            drop_cols = [c for c in interact_cols if c not in non_zero]
            X_train = X_train.drop(columns=drop_cols)
            X_test = X_test.drop(columns=drop_cols)
            #####

            # Feature selection.
            if (selected_predictors != []):
                if (use_interaction == True):

                    all_predictors = X_train.columns

                    # predictors without any interactions and that are also not
                    interaction terms.

```

```

        slicing_1 = np.sum(np.array([all_predictors.str.fullmatch(item) for
↪item in selected_predictors])), axis=0, dtype='bool')

        # all interaction terms.
        slicing_2 = all_predictors.str.startswith('pddistrict_') *
↪all_predictors.str.contains(':') \
            + all_predictors.str.startswith('intersection') *
↪all_predictors.str.contains(':') \
            + all_predictors.str.startswith('pddistrict_') * all_predictors.
↪str.endswith(':intersection') \
            + all_predictors.str.startswith('intersection:') *
↪all_predictors.str.contains('pddistrict_')

        slicing_3 = np.sum(np.array([all_predictors.str.startswith(pat =
↪item + ':') for item in all_predictors[slicing_1]]), axis=0, dtype='bool')
        slicing = slicing_1 + slicing_2 + slicing_3 # combining everything.

    else:
        slicing = X_train.columns.intersection(selected_predictors)

    if (X_train.loc[:, slicing].shape[1] > 0): # Want to avoid getting an
↪empty dataset.
        X_train = X_train.loc[:, slicing]
        X_test = X_test.loc[:, slicing]

    predictors = X_train.columns.tolist()

    if (full_preprocessing == True):

        # Rescaling the data based on the data in the train dataset.
        scaler = StandardScaler()
        scaler.fit(X_train)
        X_train = scaler.transform(X_train)
        X_test = scaler.transform(X_test)

        if (use_pca == True):
            condition = (pcs != []) & ((np.array(pcs + [0]).max()) <=
↪len(predictors))
            nbr_pcs = np.array(pcs).max() + 1 if (condition == True) else None
            pca = PCA(n_components=nbr_pcs)
            pca.fit(X_train)
            X_train = pca.transform(X_train)
            X_test = pca.transform(X_test)
            predictors = np.arange(0, X_train.shape[1]).tolist()

            if (condition == True):

                X_train = X_train[:, pcs]
                X_test = X_test[:, pcs]
                predictors = pcs

    if ((dataframe_format == True)):
        X_train = pd.DataFrame(X_train, columns=predictors, index=X_train_index)
        X_test = pd.DataFrame(X_test, columns=predictors, index=X_test_index)

    return X_train, X_test, y_train, y_test, predictors, label_enc

```

C. Predictive_algo

```
def predictive_algo(X_train, X_test, y_train, y_test,
                    learner=LogisticRegression(max_iter=1000),
                    selected_predictors=[],
                    one_hot_encoding=False,
                    drop_first=False,
                    use_interaction=False,
                    full_preprocessing=True,
                    use_pca=False,
                    pcs=[]):
    # call data_preprocessing
    processed = data_preprocessing(
        X_train=X_train.copy(), X_test=X_test.copy(),
        y_train=y_train.copy(), y_test=y_test.copy(),
        selected_predictors=selected_predictors,
        one_hot_encoding=one_hot_encoding,
        drop_first=drop_first,
        use_interaction=use_interaction,
        full_preprocessing=full_preprocessing,
        use_pca=use_pca,
        pcs=pcs,
        dataframe_format=False
    )
    Xtr, Xte, ytr, yte, predictors, label_enc = processed

    # training and scoring
    learner.fit(X=Xtr, y=ytr)
    y_pred = learner.predict(Xte)
    y_prob_pred = learner.predict_proba(Xte)

    test_acc = accuracy_score(y_true=yte, y_pred=y_pred)
    test_ll = log_loss(y_true=yte, y_pred=y_prob_pred, labels=np.
        arange(len(label_enc.classes_)))

    return {'test_acc': test_acc,
            'test_ll': test_ll,
            'y_test': yte,
            'y_pred': y_pred,
            'y_prob_pred': y_prob_pred,
            'predictors': predictors,
            'label_enc': label_enc}
```

D. Model Validation

```

def model_validation(learner=LogisticRegression(max_iter=1000),
                    split_seed=12345,
                    k=3,
                    nbr_loops=1,
                    cv_seed=54321,
                    selected_predictors=[],
                    one_hot_encoding=False,
                    drop_first=False,
                    use_interaction=False,
                    full_preprocessing=True,
                    use_pca=False,
                    pcs=[],
                    scores_only=False):

    X_train = sf_crime(train=True, split_seed=split_seed, encoding=False)
    X_test  = sf_crime(train=False, split_seed=split_seed, encoding=False)
    y_train = X_train['category']
    y_test  = X_test['category']

    # k-fold cross-validation on training set
    cv = RepeatedKfold(n_splits=k, n_repeats=nbr_loops, random_state=cv_seed)
    splitting = cv.split(X=X_train, y=y_train)

    kf_lls, kf_accs = [], []

    for train_idx, val_idx in splitting:
        X_sub_train = X_train.iloc[train_idx, :].copy()
        X_sub_val   = X_train.iloc[val_idx, :].copy()
        y_sub_train = y_train.iloc[train_idx].copy()
        y_sub_val   = y_train.iloc[val_idx].copy()

        results = predictive_algo(
            X_train=X_sub_train, X_test=X_sub_val,
            y_train=y_sub_train, y_test=y_sub_val,
            learner=learner,
            selected_predictors=selected_predictors,
            one_hot_encoding=one_hot_encoding,
            drop_first=drop_first,
            use_interaction=use_interaction,
            full_preprocessing=full_preprocessing,
            use_pca=use_pca,
            pcs=pcs
        )
        kf_lls.append(results['test_ll'])
        kf_accs.append(results['test_acc'])

```

```

kf_ll = np.mean(kf_lls)
kf_acc = np.mean(kf_accs)

# final evaluation on test set
final = predictive_algo(
    X_train=X_train, X_test=X_test,
    y_train=y_train, y_test=y_test,
    learner=learner,
    selected_predictors=selected_predictors,
    one_hot_encoding=one_hot_encoding,
    drop_first=drop_first,
    use_interaction=use_interaction,
    full_preprocessing=full_preprocessing,
    use_pca=use_pca,
    pcs=pcs
)

if scores_only:
    return {'kf_ll': kf_ll,
            'kf_acc': kf_acc,
            'test_ll': final['test_ll'],
            'test_acc': final['test_acc'],
            'predictors': selected_predictors}
else:
    print(f"k-fold log-loss : {kf_ll:.4f} | k-fold accuracy: {kf_acc:.4f}")
    print(f"Test log-loss : {final['test_ll']:.4f} | test accuracy: {final['test_acc']:.4f}")
    return {'kf_ll': kf_ll,
            'kf_acc': kf_acc,
            'test_ll': final['test_ll'],
            'test_acc': final['test_acc'],
            'y_test': final['y_test'],
            'y_pred': final['y_pred'],
            'y_prob_pred': final['y_prob_pred'],
            'label_enc': final['label_enc'],
            'predictors': selected_predictors}

```